«Подспорье» (edu-support.mephi.ru) (https://edu-support.mephi.ru)

Проект «Изображение проекции полиэдра»

Е.А. Роганов
v.5.1, 29 марта 2026 года

Построение изображения полиэдра с удалением невидимых линий — пример классической задачи, для успешного решения которой необходимо знакомство с основами вычислительной геометрии. Данный материал является первым из трёх, посвящённых этому проекту.

Постановка задачи

В настоящее время во многих областях науки и техники используются различные методы моделирования геометрических объектов в трёхмерном пространстве. Один из способов моделирования состоит в том, чтобы аппроксимировать реальный объект набором выпуклых плоских многоугольников. Такой набор мы будем называть *полиэдром*, многоугольники — *гранями*, а их стороны — *рёбрами* полиэдра. Вот формальные определения введённых терминов.

Полиэдр (polyedr)
: Множество выпуклых плоских многоугольников.

Грань полиэдра (facet)
: Выпуклый плоский многоугольник.

Ребро полиэдра (edge)
: Любая из сторон одной из граней.

Вершина полиэдра (vertex)
: Любая из вершин произвольной грани.

Примерами полиэдров могут служить поверхности куба, призмы и пирамиды (включая усечённую), листы полураскрытой веером книги и даже просто два-три плоских выпуклых многоугольника, расположенных в трёхмерном пространстве друг относительно друга произвольным образом.

При изображении полиэдра реально рисуются не сами многоугольники, а линии, их ограничивающие, то есть их рёбра. Если, однако, изобразить *все* рёбра полиэдра, то получится весьма запутанная, непривычная для человека картина. Для получения удобного для человека рисунка нужно при построении изображения учитывать, что некоторые рёбра могут оказаться полностью или частично *невидимыми*, так как их могут загородить грани полиэдра.

> Здесь следует запустить программу, которая строит изображения нескольких несложных полиэдров, и убедиться в том, насколько *неправильным* является рисование всех рёбер полиэдра.

Таким образом, при построении изображения надо *удалять* (не рисовать) части рёбер, которые не видны. По этой причине рассматриваемая задача в литературе часто называется задачей *удаления невидимых линий*.

Изображение полиэдра зависит от того, *откуда* на него посмотреть. Это направление (взгляда) должно быть задано в точной постановке задачи. Направление, противоположное направлению взгляда, будем называть *вектором проектирования* и считать, что он направлен вверх. Этого всегда можно добиться, осуществив *поворот* полиэдра вместе с заданным вектором проектирования. Для того чтобы упростить терминологию, договоримся называть в дальнейшем параллельные вектору проектирования прямые и плоскости *вертикальными*, а ортогональные ему — *горизонтальными*.

> Здесь полезно «покрутить/повертеть» простейший полиэдр.
>
> Сначала используя программу, изображающую **все рёбра**, а затем — программу, **удаляющую невидимые рёбра**.

Рассматриваемая задача, таким образом, сводится к следующей: построить проекцию полиэдра с удалением невидимых линий на *бесконечно высокую* горизонтальную плоскость. Требуемое плоское изображение при этом задано не однозначно, так как вращения вокруг вертикальной оси приводят к поворотам итоговой картинки. Будем

поэтому считать, что полная постановка задачи кроме самого полиэдра содержит также задание вращения трёхмерного пространства.

Задание полиэдра

При задании полиэдра всю информацию о нём разделяют на две части. Первая из них задаёт так называемый *абстрактный* полиэдр. При этом определяется только количество граней, рёбер и вершин в нём с указанием того, какие именно рёбра принадлежат отдельным граням, а вершины — рёбрам. Вторая часть информации, называемая *метрической*, определяет координаты всех вершин полиэдра в пространстве.

Информация, представляющая полиэдр, является достаточно объёмной, и поэтому было бы весьма неудобно вводить её, используя клавиатуру. Гораздо разумнее создать описание полиэдра в специальном файле, а программе передать его имя. Вот как будет выглядеть этот файл, представляющий поверхность единичного куба с центром в начале координат и гранями, параллельными координатным плоскостям:

```
200.0   45.0    45.0    30.0
8       6       24
-0.5    -0.5    0.5
-0.5    0.5     0.5
0.5     0.5     0.5
0.5     -0.5    0.5
-0.5    -0.5    -0.5
-0.5    0.5     -0.5
0.5     0.5     -0.5
0.5     -0.5    -0.5
4       1    2    3    4
4       5    6    2    1
4       3    2    6    7
4       3    7    8    4
4       1    4    8    5
4       8    7    6    5
```

Можете ли вы объяснить содержимое этого файла?

Первая строка содержит коэффициент гомотетии и три угла Эйлера, задающие вращение пространства (о преобразованиях пространства будет сказано чуть позже). Вторая строка говорит о том, что у куба 8 вершин, 6 граней и *24* ребра. Число 24, вдвое превышающее реальное количество рёбер у куба, является результатом того факта, что каждое из рёбер куба принадлежит ровно *двум* граням. Следующая часть файла содержит метрическую информацию — координаты всех восьми вершин полиэдра, а в заключительной части файла для каждой из шести граней куба указываются количество её вершин и перечисляются их номера.

Что нужно изменить в этом файле, чтобы куб превратился в «коробку»?

Для получения из куба открытой «коробки» достаточно удалить одну из его граней:

```
200.0   60.0    -140.0  60.0
8       5       20
-0.5    -0.5    0.5
-0.5    0.5     0.5
0.5     0.5     0.5
0.5     -0.5    0.5
-0.5    -0.5    -0.5
-0.5    0.5     -0.5
0.5     0.5     -0.5
0.5     -0.5    -0.5
4       1    2    3    4
4       5    6    2    1
4       3    2    6    7
4       3    7    8    4
4       1    4    8    5
```

Изображение проекции полиэдра без удаления невидимых линий

Математические аспекты

В рассматриваемом проекте речь идёт о геометрических объектах, находящихся в трёхмерном пространстве, поэтому нам необходимо вспомнить некоторые факты из линейной алгебры и аналитической геометрии.

Прежде всего следует вспомнить, что точку пространства $\mathbb{R}^3$ можно задать с помощью трёх действительных чисел — её координат. Эти же координаты задают и вектор трёхмерного пространства, начало которого совпадает с началом координат, а конец — с рассматриваемой точкой.

Для векторов в пространстве определены операции сложения, вычитания и умножения на число, порождающие новый вектор, координаты которого находятся с помощью сложения, вычитания и умножения на число координат

исходных векторов.

Скалярное произведение (http://ru.wikipedia.org/wiki/%D0%A1%D0%BA%D0%B0%D0%BB%D1%8F%D1%80%D0%BD%D0%BE%D0%B5_%D0%BF%D1%80%D0%BE%I
двух векторов $\vec{a}$ и $\vec{b}$, обозначаемое часто как $\vec{a}\cdot\vec{b}$, может быть записано в двух эквивалентных формах:

$$\vec{a}\cdot\vec{b} = |\vec{a}||\vec{b}|\cos\alpha = a_x b_x + a_y b_y + a_z b_z,$$

где $|\vec{a}|$ и $|\vec{b}|$ — длины векторов, α — угол между ними, а a_x, a_y, a_z и b_x, b_y, b_z — их координаты.

Наиболее часто используемыми фактами, связанными со скалярным произведением, являются следующие:

- $\vec{a}\cdot\vec{b} = \vec{b}\cdot\vec{a}$ (коммутативность);
- $|\vec{a}| = \sqrt{\vec{a}\cdot\vec{a}}$;
- $\alpha = \arccos\dfrac{\vec{a}\cdot\vec{b}}{|\vec{a}||\vec{b}|}$;
- $\vec{a}\perp\vec{b} \Longleftrightarrow \vec{a}\cdot\vec{b} = 0$;
- угол между векторами $\vec{a}$ и $\vec{b}$ острый $\Longleftrightarrow \vec{a}\cdot\vec{b} > 0$;
- угол между векторами $\vec{a}$ и $\vec{b}$ тупой $\Longleftrightarrow \vec{a}\cdot\vec{b} < 0$.

Векторным произведением (http://ru.wikipedia.org/wiki/%D0%92%D0%B5%D0%BA%D1%82%D0%BE%D1%80%D0%BD%D0%BE%D0%B5_%D0%BF%D1%80%D0%BE%D
двух векторов $\vec{a}$ и $\vec{b}$, часто обозначаемым как $\vec{a}\times\vec{b}$, называется такой перпендикулярный обоим исходным векторам вектор $\vec{c}$, длина которого равна $|\vec{a}||\vec{b}|\sin\alpha$, что вектора $\vec{a}, \vec{b}$ и $\vec{c}$ *образуют правую тройку* (это эквивалентно положительности определителя, составленного из координат данных трёх взятых в указанном порядке векторов). Вектор $\vec{c}$ можно получить, вычислив следующий определитель:

$$\vec{a}\times\vec{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}.$$

Особенно полезными являются следующие свойства векторного произведения двух векторов:

- $\vec{a}\times\vec{b} = -\vec{b}\times\vec{a}$ (антикоммутативность);
- $\vec{a}\parallel\vec{b} \Longleftrightarrow \vec{a}\times\vec{b} = 0$;
- площадь параллелограмма, построенного на векторах $\vec{a}$ и $\vec{b}$, равна длине векторного произведения этих двух векторов.

Смешанным произведением (http://ru.wikipedia.org/wiki/%D0%A1%D0%BC%D0%B5%D1%88%D0%B0%D0%BD%D0%BD%D0%BE%D0%B5_%D0%BF%D1%80%D0%BE%I
трёх векторов $\vec{a}, \vec{b}$ и $\vec{c}$ трёхмерного пространства называют скалярное произведение вектора $\vec{a}$ с векторным произведением векторов $\vec{b}$ и $\vec{c}$:

$$(\vec{a}, \vec{b}, \vec{c}) = \vec{a}\cdot(\vec{b}\times\vec{c}).$$

Смешанное произведение может быть вычислено по следующей формуле:

$$(\vec{a}, \vec{b}, \vec{c}) = \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix}.$$

Геометрический смысл смешанного произведения трёх векторов заключается в том, что его абсолютная величина равна объёму параллелепипеда, построенного на этих векторах. Как следствие, три вектора компланарны тогда и только тогда, когда их смешанное произведение равно нулю.

В данном проекте необходимо использовать линейные преобразования (отображения) (http://ru.wikipedia.org/wiki/%D0%9B%D0%B8%D0%BD%D0%B5%D0%B9%D0%BD%D0%BE%D0%B5_%D0%BE%D1%82%D0%BE%D0%B1%l плоскости и пространства. Любое такое преобразование, как известно, задаётся квадратной матрицей порядка два для плоскости и порядка три для пространства. Напомним, что тождественному преобразованию соответствует единичная матрица E.

Каков будет образ точки $(1,1,1)$ после преобразования, задаваемого матрицей $A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}$?

Среди всех преобразований нам особенно интересны повороты, о которых мы поговорим чуть позже, и *гомотетия*, задаваемая матрицей вида kE, где k — положительная константа, а E — единичная матрица. Что происходит с точками пространства при гомотетии?

Ортогональные преобразования (http://ru.wikipedia.org/wiki/%D0%9E%D1%80%D1%82%D0%BE%D0%B3%D0%BE%D0%BD%D0%B0%D0%BB%D1%8C%D0%BD%D0%BE%D плоскости и пространства, сохраняющие ориентацию, являются поворотами. На плоскости поворот на угол φ задаётся матрицей вида

$$\begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix}.$$

В случае пространства повороту на угол φ вокруг оси Oz соответствует следующая матрица:

$$\begin{pmatrix} \cos\varphi & \sin\varphi & 0 \\ -\sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Аналогичный вид имеют и матрицы поворота вокруг осей Ox и Oy, а произвольное вращение пространства может быть представлено в виде композиции трёх последовательных поворотов вокруг осей Oz, Oy и вновь Oz. Более подробно об этих так называемых трёх углах Эйлера, задающих любое вращение трёхмерного пространства, можно прочитать в соответствующей статье (http://ru.wikipedia.org/wiki/%D0%A3%D0%B3%D0%BB%D1%8B_%D0%AD%D0%B9%D0%BB%D0%B5%D1%80%D0%B0) Википедии.

В процессе работы над проектом нам потребуется решать следующую задачу: найти пересечение отрезка с полупространством, которое задано точкой на граничной плоскости и вектором внешней нормали к ней. Двумерный аналог этой задачи таков: найти пересечение отрезка на плоскости с полуплоскостью, которая задана точкой на граничной прямой и вектором внешней нормали к этой прямой. Рассмотрим решение последней из задач.

Пусть концы отрезка B (от английского слова `begin`) и F (от `final`) имеют соответственно координаты (x_1, y_1) и (x_2, y_2), точка A на граничной прямой — координаты (x_0, y_0), а вектор внешней нормали к полуплоскости — это вектор $\vec{N}$. В какой ситуации пересечение отрезка с полуплоскостью пусто?

Для этого необходимо и достаточно, чтобы оба конца отрезка лежали с «нужной» стороны от заданной прямой. Как записать это условие?

Точка (x_1, y_1) будет лежать с «нужной» (внешней по отношению к полуплоскости) стороны от граничной прямой, если угол между векторами внешней нормали и вектором, соединяющим точки (x_0, y_0) и (x_1, y_1), является острым. Как записать это условие формально?

Искомое условие имеет следующий вид: $\vec{N} \cdot \overrightarrow{AB} > 0$. Каков же ответ на исходный вопрос «в какой ситуации пересечение отрезка с полуплоскостью пусто»?

Ответ таков: $\vec{N} \cdot \overrightarrow{AB} > 0 \wedge \vec{N} \cdot \overrightarrow{AF} > 0$.

А когда пересечение отрезка с полуплоскостью представляет собой весь отрезок?

Ответ: $\vec{N} \cdot \overrightarrow{AB} < 0 \wedge \vec{N} \cdot \overrightarrow{AF} < 0$.

Если знаки указанных скалярных произведений разные, то отрезок пересекает прямую, ограничивающую полуплоскость. В этом случае для определения пересечения отрезка с полуплоскостью прежде всего необходимо найти точку их пересечения. Как?

Произвольная точка отрезка BF может быть записана как $(1-t)B + tF$, где $0 \leqslant t \leqslant 1$. При $t = 0$ эта формула задаёт начало отрезка (точку B), а при $t = 1$ — его конец (точку F). Какому значению t соответствует искомая

точка пересечения C?

Для неё $\vec{N} \cdot \overrightarrow{AC} = 0$. Отсюда $t = \frac{\vec{N} \cdot \overrightarrow{AB}}{\vec{N} \cdot \overrightarrow{AB} - \vec{N} \cdot \overrightarrow{AF}}$.

Программистские аспекты

Как уже было сказано выше, основную сложность при изображении проекции полиэдра представляет нахождение невидимых (затенённых гранями полиэдра) частей рёбер, которые не должны быть нарисованы. Несмотря на это, начать решение задачи полезно в максимально упрощённой форме — построить изображение проекции полиэдра *без удаления невидимых линий*. При этом нам придётся создать несколько классов.

Точку в трёхмерном пространстве $(\mathbb{R}^3)$, так же как и вектор этого пространства, мы будем представлять объектом класса `R3`, имеющем три действительные компоненты (`x`, `y` и `z`), — координаты точки или вектора. В этом классе реализуем ряд методов, обеспечивающих все необходимые в данном проекте манипуляции над точками (векторами).

Каждое из рёбер (экземпляров класса `Edge`) полиэдра будем задавать его началом и концом — объектами класса `R3`, а произвольную его грань (экземпляр класса `Facet`) — массивом её вершин. Сам полиэдр будет являться экземпляром класса `Polyedr`, в конструкторе (методе `__init__`) которого будет обрабатываться файл, содержащий всю информацию о полиэдре.

Мы хорошо знаем, как можно изобразить заданный отрезок на плоскости, используя модуль `TkDrawer`. Таким образом, для изображения всех рёбер полиэдра *без учёта затенения* достаточно понять, как по известным трёхмерным координатам ребра найти двумерные координаты его проекции. Как?

Так как плоскость проектирования горизонтальна, то достаточно «забыть» z-координату!

Файлы проекта

Рекомендуется очень внимательно рассмотреть приведённые ниже файлы эталонного проекта, разобраться в назначении каждого символа и задать себе массу вопросов по этому проекту, начинающихся со слов *что*, *зачем* или *почему*?

- Файл `common/r3.py`

```
from math import sin, cos


class R3:
    """ Вектор (точка) в R3 """

    # Конструктор
    def __init__(self, x, y, z):
        self.x, self.y, self.z = x, y, z

    # Сумма векторов
    def __add__(self, other):
        return R3(self.x + other.x, self.y + other.y, self.z + other.z)

    # Разность векторов
    def __sub__(self, other):
        return R3(self.x - other.x, self.y - other.y, self.z - other.z)

    # Умножение на число
    def __mul__(self, k):
        return R3(k * self.x, k * self.y, k * self.z)

    # Поворот вокруг оси Oz
    def rz(self, fi):
        return R3(
            cos(fi) * self.x - sin(fi) * self.y,
            sin(fi) * self.x + cos(fi) * self.y, self.z)

    # Поворот вокруг оси Oy
    def ry(self, fi):
        return R3(cos(fi) * self.x + sin(fi) * self.z,
                  self.y, -sin(fi) * self.x + cos(fi) * self.z)

    # Скалярное произведение
    def dot(self, other):
        return self.x * other.x + self.y * other.y + self.z * other.z

    # Векторное произведение
    def cross(self, other):
        return R3(
            self.y * other.z - self.z * other.y,
            self.z * other.x - self.x * other.z,
            self.x * other.y - self.y * other.x)
```

```
if __name__ == "__main__":  # pragma: no cover
    x = R3(1.0, 1.0, 1.0)
    print("x", type(x), x.__dict__)
    y = x + R3(1.0, -1.0, 0.0)
    print("y", type(y), y.__dict__)
    y = y.rz(1.0)
    print("y", type(y), y.__dict__)
    u = x.dot(y)
    print("u", type(u), u)
    v = x.cross(y)
    print("v", type(v), v.__dict__)
```

- Файл `common/tk_drawer.py`

```
from tkinter import *

# Размер окна
SIZE = 900
# Коэффициент гомотетии
SCALE = 1.5


def x(p):
    """ преобразование x-координаты """
    return SIZE / 2 + SCALE * p.x


def y(p):
    """" преобразование y-координаты """
    return SIZE / 2 - SCALE * p.y


class TkDrawer:
    """ Графический интерфейс """

    # Конструктор
    def __init__(self):
        self.root = Tk()
        self.root.title("Изображение проекции полиэдра")
        self.root.geometry(f"{SIZE+5}x{SIZE+5}")
        self.root.resizable(False, False)
        self.root.bind('<Control-c>', quit)
        self.canvas = Canvas(self.root, width=SIZE, height=SIZE)
        self.canvas.pack(padx=5, pady=5)

    # Завершение работы
    def close(self):
        self.root.quit()

    # Стирание существующей картинки
    def clean(self):
        self.canvas.create_rectangle(0, 0, SIZE, SIZE, fill="white")
        self.root.update()

    # Рисование линии
    def draw_line(self, p, q):
        self.canvas.create_line(x(p), y(p), x(q), y(q), fill="black", width=1)
        self.root.update()


if __name__ == "__main__":  # pragma: no cover

    import time
    from r3 import R3
    tk = TkDrawer()
    tk.clean()
    tk.draw_line(R3(0.0, 0.0, 0.0), R3(100.0, 100.0, 0.0))
    tk.draw_line(R3(0.0, 0.0, 0.0), R3(0.0, 100.0, 0.0))
    time.sleep(5)
```

- Файл `noshadow/polyedr.py`

```
from math import pi
from common.r3 import R3
from common.tk_drawer import TkDrawer


class Edge:
    """ Ребро полиэдра """
    # Параметры конструктора: начало и конец ребра (точки в R3)

    def __init__(self, beg, fin):
        self.beg, self.fin = beg, fin


class Facet:
    """ Грань полиэдра """
    # Параметры конструктора: список вершин

    def __init__(self, vertexes):
        self.vertexes = vertexes
```

```python
class Polyedr:
    """ Полиэдр """
    # Параметры конструктора: файл, задающий полиэдр

    def __init__(self, file):

        # списки вершин, рёбер и граней полиэдра
        self.vertexes, self.edges, self.facets = [], [], []

        # список строк файла
        with open(file) as f:
            for i, line in enumerate(f):
                if i == 0:
                    # обрабатываем первую строку; buf - вспомогательный массив
                    buf = line.split()
                    # коэффициент гомотетии
                    c = float(buf.pop(0))
                    # углы Эйлера, определяющие вращение
                    alpha, beta, gamma = (float(x) * pi / 180.0 for x in buf)
                elif i == 1:
                    # во второй строке число вершин, граней и рёбер полиэдра
                    nv, nf, ne = (int(x) for x in line.split())
                elif i < nv + 2:
                    # задание всех вершин полиэдра
                    x, y, z = (float(x) for x in line.split())
                    self.vertexes.append(R3(x, y, z).rz(
                        alpha).ry(beta).rz(gamma) * c)
                else:
                    # вспомогательный массив
                    buf = line.split()
                    # количество вершин очередной грани
                    size = int(buf.pop(0))
                    # массив вершин этой грани
                    vertexes = [self.vertexes[int(n) - 1] for n in buf]
                    # задание рёбер грани
                    for n in range(size):
                        self.edges.append(Edge(vertexes[n - 1], vertexes[n]))
                    # задание самой грани
                    self.facets.append(Facet(vertexes))

    # Метод изображения полиэдра
    def draw(self, tk):
        tk.clean()
        for e in self.edges:
            tk.draw_line(e.beg, e.fin)
```

- Файл `run_noshadow.py`

```python
#!/usr/bin/env -S python3 -B

from time import time
from common.tk_drawer import TkDrawer
from noshadow.polyedr import Polyedr


tk = TkDrawer()
try:
    for name in ["ccc", "cube", "box", "king", "cow"]:
        print("==============================================================")
        print(f"Начало работы с полиэдром '{name}'")
        start_time = time()
        Polyedr(f"data/{name}.geom").draw(tk)
        delta_time = time() - start_time
        print(f"Изображение полиэдра '{name}' заняло {delta_time} сек.")
        input("Hit 'Return' to continue -> ")
except (EOFError, KeyboardInterrupt):
    print("\nStop")
    tk.close()
```

- Пример файла с данными, задающими полиэдр, — файл `data/cube.geom`

```
200.0   45.0    45.0    30.0
8       6       24
-0.5    -0.5    0.5
-0.5    0.5     0.5
0.5     0.5     0.5
0.5     -0.5    0.5
-0.5    -0.5    -0.5
-0.5    0.5     -0.5
0.5     0.5     -0.5
0.5     -0.5    -0.5
4       1    2    3    4
4       5    6    2    1
4       3    2    6    7
4       3    7    8    4
4       1    4    8    5
4       8    7    6    5
```

Вопросы и задачи для самостоятельного решения

1. Создайте файлы (подобные `cube.geom`), задающие два следующих полиэдра: любая усечённая треугольная пирамида и произвольная часть единичного куба, имеющая объём 1/4, которая может быть получена в результате рассечения куба одной плоскостью, не параллельной его граням.
2. Внесите такие изменения в проект, которые позволят поворачивать уже имеющийся полиэдр произвольным образом, и получать изображение его проекции после поворота.
3. Напишите достаточно полные тесты, подтверждающие корректность методов, определённых в классе `R3`.
4. Добавьте в эталонный проект два метода, максимально эффективно вычисляющие сумму длин всех рёбер полиэдра и сумму площадей всех его граней.
5. Добавьте в эталонный проект метод, находящий центр тяжести грани полиэдра в предположении, что она изготовлена из однородного материала.
6. Добавьте в эталонный проект метод, находящий центр тяжести грани полиэдра в предположении, что вес имеют только рёбра грани, изготовленные из однородного материала.

«Подспорье» (edu-support.mephi.ru) (https://edu-support.mephi.ru)